Control-M and Kubernetes CronJob: A comparison, and addressing the feature gaps

Control-M is a commercial product with heavy infrastructure requirements. On the other hand, Kubernetes CronJob leverages existing infrastructure and achieves the same results, reducing your capex and opex.

While Kubernetes CronJob offers many benefits over traditional batch scheduling tools like Control-M, there are still some gaps that need to be addressed. Table 1 compares the key features of both.

Table 2 shows how frequently used Control-M features that are not available out-of-the-box in Kubernetes CronJob can be made available in the latter.

Let us now see how to address the gaps as quite a few features of Control-M are not available out-of-the-box in Kubernetes CronJobs.

Sending an alert on exceeding execution time

This feature of Control-M is not available out-of-the-box in Kubernetes CronJob at the time of writing this. We can create alerts in Kubernetes CronJob when execution time exceeds a certain threshold, using the Prometheus built-in monitoring and alerting system by creating metrics and the custom query for this. Here are the steps.

Set up Prometheus: Install Prometheus on the Kubernetes cluster using the Prometheus Operator or by creating a Prometheus deployment manually. Once Prometheus is running, alerts can be set up by accessing its dashboard.

Configuring CronJob metrics: Expose metrics from CronJob that Prometheus can scrape by adding annotations to the CronJob's pod template (in the Prometheus format).

```
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/path: "/metrics"
  prometheus.io/port: "8080"
```

Set up alert rules: To create an alert rule for CronJob, you need to write a PromQL query that measures the execution time of your CronJob and sets a threshold for when an alert should be triggered. Here's an example query:

```
 max by (job) (rate(my_cronjob_duration_seconds_sum[5m])) >
60
```

Control-M	How to handle in Kubernetes CronJob
Job failure alert	Create a prometheusRule that defines an alert for job failures. The rule should use the *Kube_job_status_failed* metric to trigger an alert if a job has failed.
On demand execution and restarts	1. Use the *kubectl* create job command followed by the name of the CronJob. The Kubernetes API also provides a create endpoint for the *batch/v1beta1/cronjobs/{name}/trigger* resource to trigger a CronJob using an API call. 2. To automatically restart when it fails or completes, set the *spec.concurrencyPolicy* field to 'retry', but do it after due diligence.
Job scheduling (daily, once a week, month-end, do not run on holidays, file/job trigger)	3. Kubernetes CronJob provides the ability to schedule jobs on a recurring basis using a cron expression. The cron expression consists of six fields that specify the minute, hour, day of the month, month, day of the week, and year to run the job. 4. To prevent Kubernetes CronJobs from running on holidays, use external holiday calendars or APIs to generate a list of holidays and exclude them from the schedule.
Job logs monitoring for troubleshooting (one log file per job execution)	By default, Kubernetes CronJobs do not create separate log files for each job execution. Instead, the logs for each execution are combined and stored in the pod's container log file. To troubleshoot a specific job execution, you can filter the logs by the pod name or the job name and time stamp.
Job hold (i.e., job should not run between 1 p.m. and 4 p.m.)	We can hold a Kubernetes CronJob by updating its *spec.suspend* field to true. When this field is set to true, the CronJob controller will not create new job runs and stop any active job runs.
Job notification if a job runs for more than a certain allowed SLA time limit	When the CronJob runs longer than the allowed SLA time, Prometheus will trigger the alert and send it to the configured notification system. We need to configure a Prometheus rule for this.

Table 2: How to make Control-M features available in Kubernetes CronJob